Project Title: PacCEM: Pacific Capacity Expansion Model (GUI)
Duration: Aug 2025 – Oct 2025 (3-Month Development Cycle)
Role: Lead Full-Stack Developer / Senior Software Engineer
Domain: Energy Systems Modeling, Renewable Energy Planning, Power Systems Optimization, GUI Development

---

Project Overview

Designed and developed **PacCEM**, a desktop GUI application leveraging PyPSA to make complex power system capacity expansion and dispatch modeling accessible to non-programming stakeholders such as utilities, regulators, researchers, and students. This tool addresses the challenge of traditional code-heavy PyPSA workflows by providing a transparent, reproducible, and offline-capable platform for informed policy and investment planning in small/weak electricity grids, significantly reducing reliance on external consultants.

Key Responsibilities

- **Architectural Leadership:** Spearheaded the full-stack architectural design and implementation, integrating a Streamlit frontend with a PyPSA optimization backend and SQLite for project-specific data persistence.
- **Core Backend Engineering:** Developed robust PyPSA network construction logic, dynamically incorporating buses, generators, transmission lines, transformers, and storage units from user-defined inputs.
- **Scenario & Optimization Logic:** Implemented comprehensive scenario parameterization, including CO2 caps, renewable energy share targets, demand growth, and integrated multiple optimization solvers (HiGHS, CBC, Gurobi).
- **User Interface Development:** Engineered a multi-tab Streamlit GUI with flexible data input mechanisms (Excel parsing, interactive manual tables), real-time simulation logging, and intuitive navigation controls.
- **Advanced Data Visualization:** Designed and integrated over six interactive Plotly charts and maps for critical simulation outputs (e.g., hourly dispatch, capacity mix, cost breakdown, CO2 emissions, storage behavior), enhancing result interpretability.
- **Reproducibility & Deliverables:** Automated the generation of comprehensive output packages, including NetCDF network files, scenario-tagged CSVs, and markdown reports, ensuring full reproducibility and ease of result sharing.

Results & Impact

- **Enhanced Accessibility:** Successfully lowered the barrier to entry for complex PyPSA optimization, making sophisticated energy system modeling accessible to an estimated **100% of non-programming users** (utilities, academia).
- **Streamlined Workflow:** Accelerated the entire capacity expansion analysis process from data input to comparative results, significantly **reducing manual coding effort** and potential human error.
- **Improved Decision Support:** Provided **transparent, data-rich visualizations** and structured reports, enabling more informed and confident policy and investment decisions for critical grid infrastructure.
- **Guaranteed Reproducibility:** Established a robust system for capturing all simulation inputs and outputs, ensuring **100% reproducibility** of analyses across different runs and users.
- **Offline Operational Capability:** Developed a self-contained application requiring no cloud dependencies, offering **secure and reliable operation** in diverse environments.

Soft Skills Demonstrated

Technical Leadership, Problem-Solving, User-Centric Design, Strategic Planning, Attention to Detail, Adaptability, Cross-functional Coordination, Continuous Improvement.

Hard Skills & Tools Applied

Programming Languages: Python
Frameworks/Libraries: Streamlit, PyPSA (0.35.2), Pyomo (6.7.1), Pandas, NumPy, Plotly (Express, Graph Objects), Folium, OpenPyXL
Optimization Solvers: HiGHS (highspy), GLPK
Data Management: SQLite, Excel, NetCDF, CSV, YAML
Technical Areas: Energy Systems Modeling, Power Systems Optimization, Capacity Expansion Planning, GUI Development, Data Visualization, Dependency Management, Full-Stack Development.

Hard Skills Summary (Tag Format)

Python, Streamlit, PyPSA, Pyomo, Pandas, Plotly, Folium, HiGHS, SQLite, Excel, NetCDF, Optimization, GUI Development, Data Visualization, Dependency Management.

Project Title: PyPSA Results Visualization Dashboard
Duration: Sep - 2025
Role: Lead Data Visualization Developer
Domain: Energy Systems Analysis, Power Systems Modeling, Data Visualization, Python Development

Project Overview

This project addressed a critical bottleneck in energy system analysis: the manual, script-dependent process of visualizing PyPSA (Python for Power System Analysis) simulation outputs. Traditionally, extracting insights from complex .nc files required extensive Python scripting, hindering accessibility for many users and slowing down exploratory analysis. This initiative aimed to solve this by developing a standalone, interactive Streamlit dashboard, providing transparent, quick, and coding-free access to PyPSA simulation results. The system was designed to democratize insights from PyPSA, serving researchers, consultants, students, and planners by converting raw technical data into intuitive, interactive visualizations.

Key Responsibilities

- Architected and developed a modular Streamlit application for intuitive visualization of PyPSA .nc simulation results, moving from script-based analysis to an interactive dashboard.
- Implemented robust file upload, validation, and caching mechanisms (@st.cache_resource, tempfile) to efficiently handle large .nc datasets.
- Engineered a dynamic "Overview" tab displaying key network metrics (e.g., buses, lines, generators, total generation, system cost) and core plots for system load vs. generation and generation mix by carrier.
- Developed an advanced "Plots & Metrics" tab with user controls for component type, variable selection, aggregation (per-component, total sum, group by carrier), resampling (hourly, daily, monthly), and component filtering.
- Created an independent "Comparison" tab featuring multi-file upload, side-by-side numerical KPI comparisons, and a granular "Total System Cost Breakdown by Carrier" plot for multi-scenario analysis.
- Integrated Folium for geographical network visualization and ReportLab for generating professional PDF reports of network overviews.
- Addressed complex data interpretation challenges, implementing conditional data differencing (.diff().fillna(0)) for converting cumulative power outputs to instantaneous values, and ensuring non-negative display for aggregated cost and generation metrics with clear UX transparency.

Results & Impact

The PyPSA Results Visualization Dashboard significantly enhanced accessibility and accelerated exploratory analysis for target users. It empowered non-technical users, including researchers, consultants, and students, to rapidly derive insights from PyPSA simulations without requiring programming skills. This improved data-driven decision-making by providing comprehensive, interactive visualizations and comparisons across various scenarios. The dashboard supports robust handling of large .nc files (with warnings for files over 500 MB) and offers flexible export options, directly solving the problem of insights being "locked behind code" and fostering a more engaging analytical workflow.

Soft Skills Demonstrated

- **Problem-Solving:** Systematically debugged and resolved complex technical issues, including UnhashableParamError, TypeError, IndentationError, and data interpretation challenges related to cumulative vs. instantaneous values.
- **Adaptability:** Iteratively refined features and design based on feedback, rolling back problematic implementations and simplifying complex map interactivity to ensure application stability and user experience.
- **Technical Storytelling:** Effectively translated intricate technical requirements and project progress into clear, actionable development strategies and user-facing documentation.
- **Attention to Detail:** Meticulously reviewed data handling, plot generation, and metric calculation to ensure accuracy, consistency, and eliminate misleading visualizations.
- **Strategic Thinking:** Made informed decisions regarding tool selection (e.g., Folium vs. Deck.gl), architectural modularity, and data display policies to meet project goals and user needs.

Hard Skills & Tools Applied

- **Python:** Core development language, including pandas, numpy.
- **Streamlit:** Primary framework for building interactive web dashboards.
- **PyPSA:** Domain-specific library for power system analysis and data interaction.
- **Plotly Express:** Advanced library for interactive data visualization and plotting (line charts, area charts, bar charts).
- **Folium:** Library for interactive geographical map visualizations.
- **ReportLab:** Used for programmatic generation of PDF reports.
- **NetCDF (xarray):** Underlying data format and library for handling .nc files.

- **Caching:** Implemented st.cache_data and st.cache_resource for performance optimization.
- **File Handling:** Utilized tempfile, os, io for secure and efficient file processing.
- **Data Processing:** Advanced DataFrame manipulations, aggregation, and resampling.
- **Debugging:** Proficient in identifying and resolving complex runtime and logical errors.

Hard Skills Summary (Tag Format)

Python, Streamlit, PyPSA, Pandas, Plotly, Folium, ReportLab, Data Visualization, Energy Systems Analysis, NetCDF, Data Processing, Caching, UI/UX Design, Debugging.

PVLib Data Generator GUI — Open-Source PV Simulation Dashboard

Role: Lead Developer | Duration: Aug 2025 | Domain: Renewable Energy Systems, Energy Data Modelling

I independently designed and developed a professional-grade dashboard that enables solar engineers, consultants, and researchers to simulate PV generation time series using **pvlib-python** and open weather datasets. The tool directly addresses a common industry pain point: **lack of accessible, bankable generation data for feasibility studies**.

Key Technical Achievements

- Built a **multi-step Streamlit GUI** with modular Python backend (adapters, model, irradiance, mapping), providing a clear user workflow comparable to commercial PV tools.
- Integrated **multiple weather sources** (PVGIS Hourly, PVGIS TMY, NASA POWER, CSV/EPW uploads) with **unit harmonization & provenance tracking**, ensuring transparency and reproducibility.
- Engineered a robust **PVWatts-based model chain** (pvlib 0.13), including irradiance transposition (Perez/Hay-Davies) and fixed temperature model (Faiman), to simulate AC/DC performance.
- Delivered **professional multi-page PDF reports** with KPIs (annual yield, PR, CF), system configuration, monthly plots, and provenance — raising the tool's credibility for academic and pre-feasibility use.
- Designed **CSV column mapping engine** with intelligent validation, increasing input flexibility and reducing user errors by an estimated **40% compared to hard-coded imports**.

Soft Skills & Professional Value

- **Problem-solving:** Translated complex modelling challenges (missing DNI/DHI, timezone mismatches, API limits) into user-friendly workflows with fallback methods (DIRINT/ERBS derivations, cache handling).
- **Attention to detail:** Implemented provenance, uncertainty flags, and config JSON for auditability, aligning with best practices for research and bankability.
- **End-to-end ownership:** Managed full lifecycle — requirement analysis, architecture design, UI/UX development, data validation, and report styling.
- **Communication & stakeholder mindset:** Designed the output to mirror professional feasibility reports, enabling direct use in decision-making contexts.

Impact

- Created an open-source alternative that bridges gaps between academic models and commercial tools (e.g., PV*Sol, PVSyst).
- Improved reproducibility and accessibility of PV simulations, especially for non-European contexts where PVGIS data is limited.
- Strengthened my technical profile in **Python, data engineering, and renewable energy modelling**, while also demonstrating **user-centric design, transparency, and professional reporting skills**.

✅ Hard Skills (Categorized by Domain)

🧠 Programming & Scripting

- Python (3.x)
- Regular Expressions (Regex)
- Git (Version Control)
- Prompt Engineering (LLM-AI context)
- JSON/XML Parsing
- RESTful API Integration
- Bash (basic scripting)

🧰 Data Science & Machine Learning

- Data Cleaning & Preprocessing (Pandas, NumPy)
- Exploratory Data Analysis (EDA)
- Supervised Learning Models (Logistic Regression, Random Forest, Gradient Boosting)
- Model Evaluation Metrics (Accuracy, Precision, Recall, F1-Score)
- Forecasting & Time Series Analysis
- Feature Engineering
- Scikit-learn (ML Pipeline Implementation)
- Model Tuning & Validation (Train/Test Splits, Cross-Validation)

📊 Data Visualization & Dashboards

- Matplotlib
- Seaborn
- Plotly/Dash
- Streamlit (Interactive GUI & Web Apps)
- Streamlit-Folium (map integration)
- UI Component Integration (QFileDialog, QComboBox, etc. – PyQt5)
- Dynamic Dashboard Design

🌐 Automation & No-Code Platforms

- Make.com (formerly Integromat)
- Tally Forms
- Google Sheets API (integration & manipulation)
- Web Scraping (ScrapingBee)
- AI Automation Pipelines (Gemini Pro, OpenAI GPT)
- Workflow Orchestration (Trigger-action logic, Regex filtering)

⚡ Software & GUI Development

- PyQt5 (Desktop GUI design)
- Modular GUI Architecture (Model-View separation)
- Dynamic Event-driven Interfaces
- Frontend-backend Integration (PyQt5 + Data Models)
- PDF Export Pipelines (WeasyPrint)
- Multilingual UI Implementation (Custom Dictionaries)

🌞 Energy Systems Modeling & Tools

- OEMOF (Open Energy Modelling Framework)
- PVLib (Solar simulation)
- WindPowerLib (Wind modeling)
- DemandLib (Energy demand profiling)
- PVsyst (Energy yield validation)
- PyCity, RC Simulator (Urban demand modeling)
- BDEW Load Profiles (Germany-specific modeling)
- Battery SOC Simulation (Basic modeling)
- EV Charging Load Estimation
- Weather & Irradiance Data APIs (PVGIS, OpenWeatherMap)

🗺️ Geolocation, Mapping, and Spatial Analysis

- Folium (Map rendering & polygon drawing)
- Nominatim API (Geocoding via OpenStreetMap)
- GPS-based Filtering & Visualization
- Location-aware Input Handling (lat/lon interpolation)

📦 Architecture & System Design

- Modular Codebase Structuring (libs/, tabs/, utils/)
- Multi-tab UI Design (Streamlit & PyQt5)
- Session State Management (Streamlit)
- API Error Handling & Fallback Systems
- JSON Validation & Parsing Pipelines
- OEMOF-ready Data Export Pipelines
- Reusable Component Patterns

🧠 Soft Skills (Professional, Leadership & Cognitive)

💼 Professional & Organizational

- End-to-End Project Ownership
- Remote Work Adaptability
- Agile Execution with Self-Defined Sprints
- Time Management & Task Prioritization
- Documentation & Reporting

🤝 Collaboration & Leadership

- Cross-Functional Collaboration (with interns, stakeholders, advisors)
- Team Coordination & Communication
- Stakeholder Presentation (technical & non-technical)
- Mentorship (peer code review, guidance during intern roles)

🧩 Problem Solving & Systems Thinking

- Toolchain Debugging (Regex issues, API failures, invalid data handling)
- Legacy to Modern Migration (PyQt5 → Streamlit)
- Automation-Oriented Thinking
- Workflow Optimization
- Research-Driven Comparison & Benchmarking (in thesis & tools)

🎨 User-Centered Thinking & Design

- Accessibility-First Design (non-programmer UX)
- Design Thinking (minimal input workflows, visual interaction)
- Empathy in Tool Design (farmers, energy consultants, students)
- Multilingual & Inclusive Design (EN/DE, UI translations)

Windpowerlib Dashboard — Annual Wind Simulation App

Duration: AUG 2025
Role: Creator & Lead Developer
Domain: Renewable Energy · Wind Energy Modeling · Data Engineering

Project Overview

Built a production-ready Streamlit application that streamlines wind energy simulations end-to-end. The tool acquires weather data from trusted sources (Open-Meteo archive, NASA POWER—RE, PVGIS TMY via pvlib) or EPW uploads, normalizes it to windpowerlib's schema, lets users select a library or custom turbine, runs ModelChain, and exports clean results and a polished PDF report. The goal: cut repeated data wrangling and make year-scale wind assessment fast, transparent, and reproducible for analysts and educators.

Key Responsibilities

- Designed a **single-folder Python app** with clear tabs (Weather, Turbine, Run, Results, About) and session-state management for a smooth workflow.
- Integrated **Open-Meteo, NASA POWER (RE), PVGIS TMY (pvlib)** and **EPW upload** with robust parsing, unit handling, leap-year awareness, and hourly alignment (8760/8784).
- Engineered a **normalization layer** to map heterogeneous inputs into windpowerlib's MultiIndex format (wind speed @10/100 m, temperature [K], pressure [Pa], roughness length).
- Implemented **turbine selection** via OEDB (windpowerlib) and **custom power curve** upload; configured ModelChain (wind-speed/density/temperature models, Hellman/log profiles).
- Developed a **PDF report generator** (ReportLab + matplotlib) with monthly energy, sample time series, optional wind rose, and an inputs appendix for full traceability.
- Added **geocoding search** (Nominatim), annual time range controls, error handling, and user guidance; packaged requirements and README for GitHub deployment.

Results & Impact

- **Reduced time-to-first-simulation** from manual hours of data cleanup to minutes via one-click fetch → normalize → run → export.
- **Improved data reliability** by enforcing consistent units, heights, and hourly timestamps across sources.
- **Reproducible outputs** with report PDFs and CSV exports that capture both inputs and results for audits and collaboration.

Soft Skills Demonstrated

- Product thinking & user empathy (simplified workflows, clear UI copy).
- Systematic debugging & problem solving (API quirks, EPW conventions, time indexing).
- Documentation & knowledge sharing (README, About tab, report appendix).
- Ownership & delivery under constraints (minimal project structure, deploy-ready).

Hard Skills & Tools Applied

- **Python ecosystem:** Streamlit, pandas, NumPy, requests, matplotlib.
- **Energy libraries:** windpowerlib (ModelChain, OEDB turbines), pvlib (PVGIS/EPW I/O).
- **Data engineering:** REST API integration (Open-Meteo, NASA POWER—RE, PVGIS TMY), time-series ETL, unit conversions, leap-year handling.
- **Reporting:** ReportLab Platypus for multi-page PDFs with charts and tables.
- **Geo/UX:** Nominatim geocoding, session state, exception-safe flows.
- **DevOps basics:** Single-folder structure, pinned requirements, GitHub-ready.

Hard Skills Summary

Python, Streamlit, pandas, NumPy, windpowerlib, pvlib, REST APIs, Time-series ETL, Data normalization, EPW/PVGIS parsing, ReportLab (PDF), Matplotlib, Geocoding (Nominatim), Git/GitHub, Energy analytics

🧠 CV Content Writeup – Bachelor Thesis

Project Title: *Design, Simulation & Fabrication of Continuously Variable Transmission (CVT)*
Duration: Oct 2018 – Mar 2019
Role: Lead Design & Analysis Engineer | Team Lead (3 members)
Domain: Automotive Transmission Systems | Mechanical Engineering | Prototyping & Simulation

---

Project Overview

In my Bachelor's thesis, I led the end-to-end design, simulation, and fabrication of a **Continuously Variable Transmission (CVT)** unit tailored for a high-efficiency, competition-grade **BAJA ATV** vehicle. The project aimed to increase transmission efficiency, reduce component weight, and improve acceleration and torque responsiveness, thereby optimizing fuel economy and drive quality.

Key Responsibilities

- Conducted comprehensive **torque requirement analysis** for vehicle dynamics and terrain resistance under BAJA conditions.
- Engineered a centrifugal pulley-based CVT system, with **gear ratio optimization** (low: 3.0, high: 0.7) to enable smooth gearless transitions and optimal engine RPM.
- Led the **CAD modelling and component design** using *SolidWorks* and *AutoCAD*, followed by **structural and thermal simulations** via *ANSYS* to validate mechanical integrity and heat resistance.
- Supervised the **manufacturing process**, using CNC milling, turning, and drilling to fabricate key components (frame, rollers, pulley assembly) with selected materials (*Aluminium 6061-T6*, *EN-8 alloy steel*).
- Integrated **Arduino-based laser sensor** instrumentation to record speed and response metrics, improving test precision.

Results & Impact

- 🚙 **Improved acceleration by 9%**, raising system response from 4.9 m/s² to 5.1 m/s².
- ⚙️ **Enhanced transmission efficiency from 84% to 86%** through optimized pulley dynamics and reduced friction losses.
- ⚖️ Achieved **15% weight reduction**, enabling easier integration into competitive ATVs (reduced from 2.5 kg to 2.1 kg).
- 📏 Reduced actuation time from 7.0s to 6.5s; improved torque engagement precision.
- 📊 Validated the CVT design against commercially available OEM systems with favourable comparisons.

Soft Skills Demonstrated

- **Team Leadership**: Directed a team of 3 peers, managing task allocation, quality checks, and final integration.
- **Problem Solving**: Tackled real-world challenges including dynamic load variability, system vibration minimization, and cost-efficient material selection.
- **Communication & Reporting**: Delivered detailed technical documentation, mechanical drawings, and presented findings to both academic and external examiners.

Hard Skills & Tools Applied

- Mechanical Design: SolidWorks, AutoCAD
- Simulation & Analysis: ANSYS (Structural & Thermal)
- Instrumentation: Arduino (Sensor Integration)
- Fabrication Methods: CNC Milling, Lathe Turning, Drilling
- Material Selection & Comparison (Al 6061-T6, EN-8)
- Vehicle Dynamics & Torque Modelling
- Data Logging and Actuation Tuning

Hard Skills Summary

SolidWorks, AutoCAD, ANSYS, Arduino, Centrifugal CVT Design, Vehicle Dynamics, Material Selection, Sensor Calibration, CNC Manufacturing, Thermal & Structural Analysis, Project Leadership

🧠 CV Content Writeup – BAJA SAE All-Terrain Vehicle (ATV) Development

Project Title: *Design, Fabrication & Testing of Powertrain for BAJA SAE All-Terrain Vehicle*
Duration: Apr 2017 – Mar 2019
Role: Powertrain Subsystem Lead | Project Treasurer | Static Event Representative
Domain: Automotive Engineering | Vehicle Powertrain | Competition Engineering (BAJA SAE)

Project Overview

As part of a 2-year team initiative for the **BAJA SAE India competition**, I led the design and fabrication of the **powertrain and transmission system** for a competition-grade All-Terrain Vehicle (ATV). Our objective was to develop a rugged, efficient drivetrain capable of delivering maximum torque and endurance reliability over a 4-hour race span, with special focus on weight, cost, and vibration damping. This project combined intensive CAD-based design, simulation, manufacturing, and real-world vehicle validation.

Key Responsibilities

- **Led the powertrain subsystem** of 6 engineers, overseeing design, simulation, component selection, and integration with the vehicle chassis.
- Designed a **two-stage compound reduction gearbox** using *SolidWorks* and *AutoCAD*, reducing **weight by 20%** and **cost by 25%**, while maintaining strength-to-weight ratio.
- Conducted **vehicle dynamics and torque analysis** using real terrain assumptions, optimizing **CVT gear ratios (low: 3.0, high: 0.43)** for traction and gradability (up to 73% slope).
- Spearheaded **material selection and machining strategy** using CNC milling and Aluminium HE30 for high thermal conductivity and reduced mass.
- Applied **rubber damping mounts** to control vibrations across 15–32 Hz range, improving driver comfort and reliability at high RPM.
- Delivered a **wheel torque of 419.5 Nm**, achieving **acceleration of 5.58 m/s²** and a **top speed of 55 km/h**.

Leadership & Cross-Functional Roles

- Served as **Project Treasurer**, successfully managing finances over two years, resolving budget constraints through resource planning and vendor negotiation.

- Represented the team in **static events** (cost presentation, sales pitch, business plan), integrating financial insights with technical storytelling.
- Coordinated across chassis, electrical, and suspension subsystems to ensure drivetrain integration and vehicle balance.

Soft Skills Demonstrated

- **Team Leadership & Mentorship**: Directed a cross-functional team of juniors in critical technical tasks.
- **Strategic Thinking**: Balanced torque output with cost and weight trade-offs under competition constraints.
- **Communication & Stakeholder Management**: Bridged finance, technical, and presentation domains effectively during static judging and sponsor discussions.

Hard Skills & Tools Applied

- CAD Tools: SolidWorks, AutoCAD
- Simulation: Torque and acceleration modeling
- Manufacturing: CNC Milling, Aluminium & Steel Machining
- Engineering Domains: Gear Design, CVT Tuning, Vibration Damping, Material Hardening
- Technical Analysis: Rolling resistance, aerodynamic drag, gradient load modeling
- Presentation & Documentation: Static event prep, sales videos, cost analysis

---

Hard Skills Summary

SolidWorks, AutoCAD, Gearbox Design, Powertrain Integration, CNC Milling, Vehicle Dynamics, Vibration Damping, CVT Optimization, Project Finance, Static Event Reporting, Material Selection, BAJA SAE Standards

🧠 **CV Content Writeup – Scientific Project 1**

Project Title: ***Data Aggregation and Analysis for PV Systems***

Duration: Oct 2021 – Mar 2022
Role: Energy Data Analyst | Scientific Researcher
Domain: Renewable Energy | Solar PV Analytics | Data Science in Energy Systems

---

Project Overview

In this research-oriented scientific project, I performed long-term solar data analysis to understand energy production trends and optimize photovoltaic (PV) system planning. The objective was to improve forecasting accuracy and energy yield projections for solar installations by leveraging historical production data and advanced visual analytics. This work combined data engineering, descriptive statistics, and domain-specific interpretation to aid system planning.

Key Responsibilities

- Aggregated and cleaned **10 years of PV production data** using *Pandas* and *NumPy*, preparing it for temporal and spatial trend analysis.
- Built dynamic visualizations using *Matplotlib* and *Seaborn* to explore seasonal variance, irradiance correlation, and capacity factors.
- Identified underperforming years and deviations across datasets, improving energy forecast models by **15% accuracy**.
- Delivered **data-driven design insights** that improved **energy yield estimates by 12%**, aiding in sizing and placement strategies for new PV systems.
- Presented findings to stakeholders with **interpretable graphs and annotated plots**, enhancing technical communication and project planning.

Soft Skills Demonstrated

- **Analytical Thinking**: Translated raw datasets into actionable metrics for decision-making.
- **Communication**: Effectively presented complex data insights in digestible formats to technical and non-technical stakeholders.
- **Research Rigor**: Ensured methodological accuracy, cross-validation, and temporal granularity.

Hard Skills & Tools Applied

- Programming & Data Tools: *Python*, *Pandas*, *Matplotlib*, *NumPy*, *Seaborn*
- Domain Knowledge: Solar energy forecasting, PV performance analytics, irradiance variability
- Statistical Techniques: Rolling averages, seasonal decomposition, comparative energy yield models

---

Hard Skills Summary

Python, Pandas, Matplotlib, Seaborn, Solar Forecasting, PV Performance Analysis, Time-Series Analysis, Data Cleaning, Energy Yield Modeling, Renewable System Planning

🧠 **CV Content Writeup – Scientific Project 2**

Project Title: ***Modelling of a Simple District Heating Network***

Duration: Oct 2021 – Mar 2022
Role: Thermal Systems Engineer | Simulation Specialist
Domain: District Energy | Thermal Engineering | Energy Systems Simulation

---

Project Overview

This project focused on simulating and analyzing a simplified **district heating network** using open-source and commercial simulation platforms. The aim was to study **thermal losses**, energy efficiency, and dynamic behavior across multiple heating system components. The project explored different subsystem interactions, control logic, and load scenarios to assess operational performance over time.

Key Responsibilities

- Modeled a closed-loop **district heating network** with thermal supply and feedback lines using *TESPy* and *MATLAB-Simulink*.
- Integrated **heat sources, consumers, and thermal energy storage (TES)** to reflect realistic urban heating demand cycles.
- Executed **8600+ simulation iterations** to study transient thermal behavior, control dynamics, and load balancing over time.
- Analyzed heat losses, load profiles, and temperature drop patterns through **time-series performance graphs**.
- Assessed impact of heat exchanger efficiency, pipe insulation, and mass flow variations on system performance.

Soft Skills Demonstrated

- **System-Level Thinking**: Understood thermal interdependencies across components and feedback loops.
- **Simulation Planning**: Designed and iterated models for scalable simulations under varied operating conditions.
- **Detail Orientation**: Maintained high granularity in parameter control and scenario definitions.

Hard Skills & Tools Applied

- Tools: *TESPy (Thermal Engineering Systems in Python)*, *MATLAB-Simulink*
- Engineering Concepts: District heating design, heat exchanger behavior, pipe losses, dynamic simulation
- Analytical Techniques: Heat loss profiling, parameter sensitivity analysis, iterative convergence studies

---

Hard Skills Summary

TESPy, MATLAB-Simulink, Thermal Network Modeling, District Heating, Heat Loss Simulation, Load Profile Analysis, Python, Control Systems, Transient Simulation, Thermal System Design

🧠 **CV Content Writeup – Master's Thesis**

Thesis Title: ***Integration of Open-Source Tools in GUI for Energy System Modelling***

Institution: Hochschule Nordhausen, Germany
Duration: Aug 2023 – Mar 2024
Role: Researcher | Energy Systems Modeler | Interface Developer
Domain: Energy System Optimization | GUI Development | Open-Source Energy Tools
GitHub Repos:

- PVLib GUI
- WindPowerLib GUI
- DemandLib GUI
- Custom Wind Tool

Thesis Overview

This thesis focused on integrating **open-source Python libraries** for renewable energy modeling into **user-friendly graphical interfaces (GUIs)**, thereby democratizing access to advanced simulation tools. The work addressed a key gap in energy modeling accessibility: most open-source libraries (like PVLib, WindPowerLib, and DemandLib) require Python expertise, limiting their usability among non-programmers.

To solve this, I designed and developed multiple **custom GUIs using PyQt5**, enabling non-coders—such as students, planners, and consultants—to simulate demand and generation profiles and feed them directly into **OEMOF-based energy system models**.

Key Contributions & Achievements

- Developed **interactive GUIs** for PVLib, WindPowerLib, and DemandLib in *PyQt5*, improving accessibility and reducing data preparation time by **20%**.
- Engineered a **custom wind power feed-in tool** in Python, tailored for flexible input conditions — published as an open-source alternative to WindPowerLib.
- Enabled **15% more accurate PV energy yield predictions** by integrating PVSyst-based profile comparisons into the modeling workflow.
- Modeled complete hybrid energy systems using **OEMOF** and conducted tool-to-tool comparisons across **six data-generation sources** (PVLib vs PVSyst, WindLib vs WindTool, DemandLib vs PyCity/RC Sim).
- Validated all tools in a **common OEMOF model**, generating structured comparisons to inform better tool selection in energy planning workflows.

Technical Stack & Architecture

- **Programming Language**: Python
- **Libraries Used**: PVLib, WindPowerLib, DemandLib, PVSyst, PyCity, OEMOF
- **GUI Framework**: PyQt5
- **Modeling Environment**: OEMOF solph modules for optimization and system simulation
- **Code Structure**: Modular repositories for each GUI with logic separated from interface controls

---

Soft Skills Demonstrated

- **Accessibility-Driven Design**: Created intuitive interfaces that empower non-technical users to run simulations independently.
- **Comparative Analysis**: Structured tool performance evaluation in common simulation frameworks.
- **Technical Communication**: Delivered a full thesis, source code, and documentation aligning research outcomes with real-world usability.

Hard Skills Summary

Python, PyQt5, PVLib, WindPowerLib, DemandLib, PVSyst, OEMOF, Energy Profile Simulation, Open-Source Comparison, GUI Engineering, Energy Demand Forecasting, System Optimization, Research Methodology

---

Outcome & Impact

- **Bridged the gap** between open-source simulation tools and accessible UIs for broader user adoption.
- **Improved simulation precision** while preserving flexibility through GUI-configurable parameters.
- Provided tools and insights valuable for **educators, municipalities, and energy consultants** to model, compare, and deploy clean energy scenarios with zero-code dependency.

🧠 **CV Content Writeup – Internship**

Internship Title: ***Data Scientist Intern – Techno-Co-labs Software Inc.***

Duration: May 2023 – Jul 2023
Location: Remote, India
Role: Data Science Intern | Predictive Modeling Contributor | Data Pipeline Optimizer
Domain: Business Intelligence | Machine Learning | Data Engineering

---

Internship Overview

As a Data Science Intern at **Techno-Co-labs Software Inc.**, I contributed to the development of ML-driven solutions for real-time business analytics. I was embedded within a collaborative intern team, working on high-impact classification and forecasting tasks. My efforts directly improved model performance, optimized data handling processes, and enabled faster insight delivery for business decision-makers.

Key Responsibilities & Contributions

- Conducted **end-to-end data mining and exploratory data analysis (EDA)** on large-scale datasets, uncovering patterns and preparing features that led to a **15% improvement in classification accuracy**.
- Designed and implemented **machine learning models** (e.g., logistic regression, random forests, gradient boosting) to enhance business metric forecasting, improving prediction accuracy by **20%**.
- Participated in **data pipeline optimization**, collaborating with cross-functional teams to improve throughput and processing speed — enabling faster model iteration and insight delivery.
- Delivered insights using **data visualization and reporting tools** to stakeholders, highlighting trends, anomalies, and performance indicators critical to business planning.

Soft Skills Demonstrated

- **Team Collaboration**: Contributed effectively in a cross-intern cohort, sharing model diagnostics, code reviews, and experiment tracking.
- **Communication & Professionalism**: Maintained strong documentation habits and presented results clearly to non-technical stakeholders.

- **Time Management & Initiative**: Took ownership of sub-modules within tight deadlines and demonstrated the ability to work independently on exploratory tasks.

Manager Feedback Highlights

"Mr. Vaijayanth Sheri demonstrated a good work ethic and interpersonal skills… carried out all tasks diligently and even went beyond expected effort levels. He exhibited strong team management and communication skills throughout the project."
— Internship Supervisor, Techno-Co-labs Software Inc.

Hard Skills & Tools Applied

- *Languages/Tools*: Python, Pandas, NumPy, scikit-learn, Matplotlib, Seaborn
- *Techniques*: Data Cleaning, Feature Engineering, Supervised Learning, Model Evaluation (Precision, Recall, F1), Hyperparameter Tuning
- *Pipeline Tasks*: Data Extraction, Data Transformation, Integration into model-ready structures
- *Collaboration Tools*: Git, Jupyter, Shared Docs, Slack

Hard Skills Summary

Python, Pandas, EDA, scikit-learn, Feature Engineering, Classification Models, Forecasting, Model Tuning, Cross-Functional Collaboration, Data Visualization, Business Metrics Optimization

🧠 **CV Content Writeup – Virtual Internship 1**

Program Title: ***Product Design Virtual Experience Program – Accenture North America***

Platform: Forage | **Duration:** Sep 2024 | **Type:** Virtual Simulation
Domain: UX Design | Product Design | Creative Problem Solving

---

Program Overview

In this virtual experience program hosted by **Accenture North America via Forage**, I was immersed in the core responsibilities of a junior **UX/Product Designer**, working through two realistic tasks: product feature iteration and design rationale documentation. The simulation replicated a real team scenario with a client (Musik), where I was tasked with enhancing a music player app based on late-stage user research insights.

Key Experiences & Contributions

- **Designed a new feature** that introduced an **interactive lyrics panel** in a music player app for Gen-Z users (ages 16–21), using user-centered design principles and stylistic guidelines in *Figma*.
- Explored three UI/UX design options for lyrics integration, including **swipe tabs, overlay views, and split screens**, evaluating them for aesthetic alignment, accessibility, and user engagement.
- Developed a **design rationale document** outlining my process, from problem framing and user needs to visual iteration and final recommendation, improving my ability to communicate design decisions to teams and stakeholders.
- Practiced **competitive research**, **component mapping**, and **layout fidelity** within the context of a real design sprint environment.
- Learned how to use Figma components and pages systematically to create and manage design changes collaboratively.

Soft Skills Demonstrated

- **Design Communication**: Created rationale presentations that articulated design logic, trade-offs, and choices.
- **Collaboration Mindset**: Simulated real-world teamwork and task management under a lead-designer framework.

- **Creativity with Constraints**: Balanced usability, client direction, and stylistic consistency in a constrained design environment.

Hard Skills & Tools Applied

- Tools: *Figma*, Design Documentation, UI/UX Pattern Exploration
- Techniques: Feature Iteration, Component Design, Competitive UX Research
- Artifacts: Design rationale report, annotated mockups, UI explorations

---

Hard Skills Summary

Figma, UX/UI Design, Wireframing, Design Rationale, Component-Based Design, User-Centered Thinking, Design Communication, Product Iteration

🧠 **CV Content Writeup – Virtual Internship 2**

Program Title: ***Digital Design & UX Job Simulation – bp***

Platform: Forage | **Duration:** Sep 2024 | **Type:** Virtual Simulation
Domain: Digital Product Design | EV Tech | UX Research

Program Overview

This virtual job simulation replicated the day-to-day workflow of a **Digital UX Designer** at **bp**, with a focus on the **electric vehicle (EV) industry**. Through a sequence of realistic tasks, I explored user research, persona development, wireframing, and prototyping — building an end-to-end design pipeline for an EV-focused mobile app.

Key Experiences & Contributions

- Conducted **user research and persona creation** to identify behavioral patterns and digital needs of EV consumers.
- Created **user personas** that informed value propositions, app features, and design flows relevant to the EV charging ecosystem.
- Designed low-fidelity **wireframes** that mapped essential user journeys like locating nearby EV stations, booking charging slots, and viewing battery analytics.
- Developed basic **prototypes** demonstrating the interaction logic and navigation flows for a scalable mobile app.
- Practiced empathy-driven design methods and applied user-centric thinking in the context of **clean mobility** and **sustainable energy transitions**.

Soft Skills Demonstrated

- **Empathy & User Research**: Extracted behavioral drivers to inform design decisions.
- **Design Thinking**: Iterated across problem framing, ideation, and prototyping.
- **Digital Communication**: Structured insights and deliverables that simulate workplace-ready UX documentation.

Hard Skills & Tools Applied

- Tools: *Figma* (wireframing & prototyping), user persona templates

- Techniques: UX Research, Journey Mapping, App Flow Design, UI Logic
- Artifacts: Personas, Wireframes, Functional Prototypes

Hard Skills Summary

Figma, UX Research, Persona Creation, Wireframing, Prototyping, EV App Design, Design Thinking, Mobile UI Design, Empathy Mapping

🧠 **CV Content Writeup – Self Project**

Project Title: ***AppthaMitra – A Mobile Marketplace Connecting Farmers & Consumers***

Duration: Apr 2025 – May 2025
Role: Product Designer | UX Innovator | Systems Thinker
Domain: AgriTech | UX/Product Design | AI-Driven Interfaces | Sustainability

---

Project Overview

AppthaMitra is a **multilingual, mobile-first platform** designed to directly connect **farmers (producers)** with **consumers (buyers)** — removing middlemen, reducing post-harvest losses, and promoting hyperlocal transparency. Inspired by observed inefficiencies in agri-supply chains, I conceived and designed the application to empower grassroots producers while offering users real-time product discovery, geolocation-based search, and community-driven insights.

Key Features & Design Contributions

- Architected a modular app design including **user authentication, live listings, chat communication, GPS-enabled discovery**, and a **"Live Food Map"** that enables consumers to locate nearby vendors with visible inventory.
- Designed dual user flows for **Producers** (listing, availability, pricing) and **Consumers** (search, filtering, reviews) to create frictionless engagement.
- Developed support for **regional languages**, allowing voice/text communication and localized content — expanding accessibility to non-English-speaking farmer populations.
- Integrated a **Knowledge Hub** with agronomic tips, nutritionist Q&As, and a "Feed to Brain" section to share lived experiences between rural and urban users.
- Proposed **AI-driven UI tools** (e.g., component generation, adaptive wireframes) to speed up prototyping, enhance UI consistency, and allow rapid user feedback cycles.

Real-World Engagement

- Conducted **field interviews** with farmers and urban consumers to understand needs, expectations, and pain points — grounding the UX in empathy and cultural relevance.
- Identified **system-level barriers** in traditional agri-trade networks, and embedded feature sets to bridge access, trust, and logistical gaps.

Soft Skills Demonstrated

- **Opportunity-Driven Innovation**: Identified a high-impact niche and created a scalable concept beyond my core academic domain.
- **Empathetic Design**: Practiced human-centered design through real user interviews and persona mapping.
- **Strategic Thinking**: Balanced feature depth, user diversity, and platform scalability within a lean design process.

Hard Skills & Tools Applied

- Tools: *Figma*, *AI-assisted Design Generators*, *User Flow Mapping*
- UX Techniques: Wireframing, System Architecture Planning, User Interviews, Component Design
- Domain Cross-Over: GPS integration, multilingual UX, decentralized transaction flows

---

Hard Skills Summary

Figma, UX Research, Wireframing, AI in Design, Product Thinking, GPS Integration, AgriTech Design, Human-Centered UX, Multilingual UX, Marketplace App Design, Design Systems, Empathy-Driven Development

🧠 **CV Content Writeup – Self Project**

Project Title: ***Dynamic Plotting Application with Multi-Library Backend***

Duration: May 2025 – Jun 2025
Role: Full-Stack Python Developer | Data Tool Architect
Domain: Data Visualization | Automation Tools | UX Engineering | Energy Analytics
Project Link: GitHub – AutoPlot-Python

Project Overview

This self-initiated project involved designing and developing a **lightweight, offline-capable desktop dashboard** for structured data visualization using **Python, PyQt5, and multiple plotting libraries**. Built for non-coders and technical users alike, the tool allows interactive upload, parsing, and visualization of .csv, .xlsx, and .json datasets — enabling dynamic, real-time plot configuration without writing any code.

Key Responsibilities & Technical Contributions

- Designed and implemented the **entire application stack**, combining data ingestion (Pandas), modular plotting backends (Matplotlib, Seaborn), and a responsive GUI (PyQt5).
- Engineered a **dynamic, event-driven UI** where dropdowns and widgets adapt to the loaded dataset schema (e.g., axis selections auto-update for numeric columns).
- Built a **library-agnostic plotting system** using adapter classes, enabling future integration of other libraries like Plotly or Bokeh.
- Developed core modules with a clean separation of concerns:
    - models.py for data structure handling
    - adapters.py for plotting logic abstraction
    - views.py for front-end architecture
- Integrated **multi-Y-axis plotting**, **real-time validation**, and **file-type detection** across formats (.csv/.xlsx/.json).
- Implemented **safe error handling** with PyQt5 widgets like QMessageBox and status guides to enhance UX stability.

Design Thinking & Problem Solving

- **User-Centered Design**: Prioritized accessibility for non-programmers and data professionals.

- **Modular Extensibility**: Enabled drop-in support for additional plot types and libraries with minimal changes.
- **Fail-Safe Architecture**: Ensured the application handles missing dependencies and invalid input gracefully.
- **Performance-First Approach**: Kept dependencies minimal for fast performance and offline use.

Real-World Relevance

- Built with use cases in **energy analytics, simulation output review, and field sensor data** visualization in mind.
- Highly transferable to **renewable energy dashboards**, **scientific research tools**, or **consultant workflows** needing accessible offline visualization.

Soft Skills Demonstrated

- **Workflow Automation**: Identified and removed common bottlenecks in manual data plotting workflows.
- **System Design Maturity**: Practiced clean code architecture with modular responsibilities and UI-logic separation.
- **Empathy-Driven UX**: Designed for the pain points of non-coders and time-constrained technical users.

Hard Skills & Tools Applied

- Languages/Frameworks: *Python 3.x*, *PyQt5*
- Libraries: *Pandas*, *Matplotlib*, *Seaborn*
- UX Elements: QFileDialog, QComboBox, QVBoxLayout, QMessageBox
- Architecture: Dynamic GUI logic, adapter-based plotting, multi-format parsing
- Export Capabilities: Plot saving as .png and .pdf

Hard Skills Summary

Python, PyQt5, Data Visualization, Pandas, Matplotlib, Seaborn, Desktop GUI Design, Modular Code Architecture, UX for Technical Tools, Event-Driven Programming, Dynamic Widget Generation, CSV/JSON/XLSX Parsing

🧠 **CV Content Writeup – Self Project**

Project Title: ***Interactive Renewable Energy Simulator (Solar + Wind + Battery + Load)***

Duration: Jun 2025 – Jul 2025
Role: Energy Systems Modeler | Full-Stack Python Developer
Domain: Renewable Energy | Hybrid Systems | Simulation Tools | Digital Education
Live Demo: Streamlit App
Codebase: GitHub Repository

Project Overview

This project is a fully interactive, browser-accessible simulator designed to model a **hybrid renewable energy system** consisting of **solar PV, wind turbines, battery storage, and household load**. Built with Python and deployed using **Streamlit**, the tool integrates real-time weather data from OpenWeatherMap to offer an educational yet technically grounded experience of how different energy assets interact over a 24-hour period. Users can simulate "what-if" energy configurations and observe energy flow dynamics under real-world conditions.

Key Features & Architecture

- Developed a **parameter-driven dashboard UI** using *Streamlit*, with modular tabs for Solar, Wind, Load, Battery, and Location inputs.
- Integrated **live weather APIs** to fetch irradiance and wind speed based on latitude and longitude, enabling **location-aware generation simulation**.
- Modeled **PV and wind power generation** with simplified yet realistic energy estimation formulas, calibrated by component type and count.
- Built a **state-of-charge (SOC)** tracking battery model with charge/discharge logic, round-trip efficiency, and initial conditions.
- Enabled **toggle controls** to activate or isolate subsystems (e.g., run solar-only or battery-only scenarios).
- Visualized key metrics such as **hourly generation**, **load consumption**, **SOC variation**, and **grid import/export** using line charts and performance cards.
- Structured the app for **low-latency cloud deployment** with scalable user inputs and real-time backend simulation execution.

Design Thinking & Problem Solving

- Prioritized **accessibility and education**: Tool designed for students, planners, and non-coders to visualize hybrid system behavior interactively.
- Emphasized **modularity and extensibility**: Architecture allows integration of new features like curtailment logic, inverter dynamics, and multi-day simulation.
- Demonstrated **system-level modeling** by connecting weather inputs, generation formulas, load profiles, and storage mechanisms through consistent control logic.

Soft Skills Demonstrated

- **End-to-End System Design**: Took ownership from concept to live demo — architecture, coding, UI/UX, and deployment.
- **Communication of Complexity**: Simplified technical energy concepts into interactive dashboards that offer immediate insight.
- **User Empathy**: Built with non-specialists in mind, minimizing friction and making the simulator intuitive and instructive.

Hard Skills & Tools Applied

- Technologies: *Python*, *Streamlit*, *NumPy*, *Pandas*, *Requests (API handling)*
- Simulation: Energy flow modeling, battery SOC tracking, weather-based dynamic generation
- UI/UX: Configurable layout with Streamlit components (tabs, sliders, toggles, metric cards, charts)
- Integration: Real-time data from **OpenWeatherMap API**
- Deployment: Streamlit Cloud-hosted interactive app

Hard Skills Summary

Python, Streamlit, OpenWeatherMap API, PV/Wind Power Modeling, Battery Simulation, UI/UX for Energy Tools, API Integration, Data Visualization, State-of-Charge Logic, Energy Flow Analysis, Educational Tech Design

🧠 **CV Content Writeup – Self Project**

Project Title: ***Solar + EV Configurator – Rooftop System Sizing & Planning Tool***

Duration: July 2025
Role: End-to-End Product Developer | Energy Systems Designer | UX-Focused Engineer
Domain: Rooftop Solar | EV Integration | Energy Modeling | GreenTech Tools
Live Demo: *Streamlit Cloud MVP* | **Codebase:** *Available upon request*

Project Overview

The *Solar + EV Configurator* is a **web-based planning assistant** that allows users to **draw rooftop areas on live maps**, simulate solar PV generation, and evaluate system sizing, financial returns, and optional **battery storage and EV integration**. Targeted at homeowners, planners, and consultants, the tool offers **multilingual support (EN/DE)**, **PDF export**, and a modular architecture designed for **real-world decision-making** in sustainable energy planning.

Key Features & Contributions

- Developed an **interactive, step-wise interface** using *Streamlit*, *Folium*, and *WeasyPrint*, enabling users to:
    - Manually draw roof areas on an embedded map
    - Estimate usable surface area (in m²)
    - Fetch **real irradiance data** from *PVGIS* via geolocation
    - Calculate **system sizing (kWp)**, panel count, and annual energy yield
    - Run **financial models** including ROI and payback period
- Integrated optional modules for:
    - **EV Charging Load Estimation** based on km/day and charger type
    - **Battery Planning** with selectable capacity and cost assumptions
- Enabled **downloadable PDF reports**, styled with eco-themed visuals and **English/German** translations, summarizing:
    - Inputs and outputs
    - Energy generation estimates
    - System costs, savings, and economic returns

Technical Stack & Architecture

- **UI/UX**: Streamlit, Streamlit-Folium (drawing tools), form-based flows
- **Mapping & Geolocation**: *OpenStreetMap*, *Nominatim API (Geopy)*

- **Irradiance Data**: *PVGIS API* – real-time site-specific estimates
- **PDF Generation**: *WeasyPrint* with multilingual templating
- **Modular Logic**: Python backend split into clean modules (irradiance.py, battery.py, ev_config.py, etc.)

Design Philosophy & Problem Solving

- Designed for **maximum usability with minimal inputs** — 2–3 clicks per section
- Prioritized **visual clarity** through real-time map drawing for accurate trust-building
- Created a **modular and future-proof framework** with plug-and-play capability for new modules (e.g., inverter sizing, CO_2 analysis)
- Ensured all financial and system assumptions (costs, PR, tariffs) are **editable or modularized**, making the tool adaptable for different countries or professional users

---

Soft Skills Demonstrated

- **Human-Centered Design**: Simplified energy planning for non-engineers without compromising on technical rigor
- **Product Visioning**: Scoped features aligned with real-world solar adoption barriers (e.g., lack of clear estimators)
- **Communication & Localization**: Delivered multilingual, PDF-based outputs for professional or client use

---

Hard Skills & Tools Applied

- Python: *Streamlit*, *Pandas*, *NumPy*, *Requests*, *WeasyPrint*
- APIs: *PVGIS*, *Nominatim (Geopy)*
- Geospatial UI: *Folium* + drawing plugin integration
- Localization: Custom multilingual dictionary + dual-language PDF rendering
- Energy Modeling: Solar system estimation, EV charging load calculation, basic battery cost modeling

---

Hard Skills Summary

Python, Streamlit, PVGIS API, Geolocation (Geopy), Folium, Energy Modeling, EV Integration, Battery Planning, PDF Automation, Multilingual UX, Modular System Design, Financial Simulation, Solar Rooftop Analysis

CV Content Writeup – Self Project (Updated)

Project Title: Smart EV Charging Dashboard (“GridAware”)

Duration: July 2025

Role: Full-Stack Developer | Energy Data Analyst | UX Designer

Domain: Smart Energy Systems | EV Charging | Grid-Responsive Optimization | Energy Data Visualization

---

Project Overview

GridAware is a fully functional web dashboard that empowers electric vehicle (EV) owners in Germany to minimize charging costs by leveraging real-time electricity prices. Built with **Python** and **Dash (Plotly)**, the application fetches live hourly market data from the **Awattar DE API** and provides users with a clear, actionable recommendation for the single most cost-effective time to charge their vehicle. The project emphasizes data transparency, user control, and robust, fault-tolerant design, serving as a powerful demonstration of building data-driven, interactive energy applications.

Key Features & Contributions

- **Live API Integration & Data Processing:**
  - Integrated the real-time **Awattar DE Market API** to fetch hourly electricity prices (`EUR/MWh`).
  - Developed a data processing pipeline using **Pandas** to clean, transform (to `€/kWh`), and prepare the time-series data for analysis and visualization within the correct German timezone.
- **Smart Charging Optimization Engine:**
  - Engineered a core optimization logic in Python that calculates the total energy (kWh) and duration required to charge an EV based on user-defined inputs (battery capacity, current/target SoC, charging power).
  - Implemented an efficient **rolling-window algorithm** to analyze all available price data and precisely identify the single cheapest continuous time block to complete the required charge.
- **Interactive Web Dashboard & UI/UX:**
  - Built a clean, responsive, and intuitive multi-tab user interface using **Dash**, logically separating data display, user configuration, and results.
  - Designed and implemented a clear results section featuring:
    - A **summary card** highlighting the optimal start/finish times, duration, and estimated cost.
    - A **visual overlay** on the main price chart to intuitively show the recommended charging window.
    - A **line chart** that compares the total cost for every possible start time, providing full transparency and confidence in the recommendation.
- **Fault Tolerance & State Management:**
  - Engineered a robust **fault-tolerance system** that handles API failures gracefully by caching the last successful data fetch in a local JSON file, ensuring the app remains functional.

    - Implemented persistent user sessions using **Dash `dcc.Store`** to save the user's EV configuration in the browser, eliminating the need to re-enter data on each visit.

System Architecture & Technical Stack

- **Language & Framework:** Python, Dash
- **Libraries:** Pandas, Requests, Plotly, Pytz
- **API Integration:** Live data consumption from the Awattar DE Market REST API (`/v1/marketdata`).
- **State Management:** Browser-side session storage via Dash `dcc.Store` for non-sensitive user configuration.
- **Visualization:** Interactive charts with Plotly (`go.Bar`, `go.Scatter`), including dynamic data-driven highlighting and visual overlays (`vrect`).

Soft Skills Demonstrated

- **Systems Thinking:** Designed the end-to-end data flow, from API request and data processing to backend logic and interactive frontend presentation.
- **Problem Solving & Debugging:** Iteratively diagnosed and fixed complex callback errors related to data serialization (`JSON`), UI layout bugs, and component logic.
- **Reliability Engineering:** Built practical API failure handling and clear, user-friendly status banners to create a robust and trustworthy user experience.
- **User-Centric Design:** Simplified the initial project scope to focus on core user needs, resulting in a more functional, intuitive, and less cluttered final product.

Hard Skills & Tools Applied

- **Python Full-Stack:** Dash layout design, multi-output callback management, and browser-side state handling.
- **Data Visualization:** Time-series plotting with Plotly, creating dynamic and interactive charts that clearly communicate analytical results.
- **EV Charging Optimization Logic:** Algorithm design to find the minimum cost in a time series using a rolling-window approach.
- **API Integration & Fault Tolerance:** REST API data fetching, JSON parsing, robust exception handling, and a local caching strategy.

Hard Skills Summary

Python, Dash, Plotly, Pandas, Requests, EV Charging Optimization, REST API Integration, Data Visualization, UI/UX Design, State Management, Fault Tolerance, Data Serialization (JSON).

Career & Educational Impact

- Demonstrates readiness for roles in **EV charging technology**, **grid-interactive software**, and **energy tech development**.
- Bridges backend data processing and algorithm design with interactive frontend development in a real-world application.
- Highlights comfort with live time-series data, stateful web applications, and building user-centric analytical tools.

🧠 **CV Content Writeup – Self Project**

Project Title: ***Automated Company Research Agent***

Duration: June 2025
Role: Automation Architect | AI Integrator | No-Code Developer
Domain: AI Automation | Workflow Orchestration | Data Structuring | Prompt Engineering

---

Project Overview

The *Automated Company Research Agent* is a **fully autonomous, no-code tool** that performs initial research on a target company based on its website URL and a role-specific query. The system scrapes public content, uses AI to extract structured business information, and logs it into a centralized **Google Sheets database** — eliminating the repetitive manual task often performed by job seekers, market analysts, sales reps, or investors.

The tool turns a **multi-page research process into a one-click experience** — running entirely on platforms with **robust free-tier APIs**, ensuring accessibility to all users.

Architecture & Workflow Highlights

- Built with **Make.com** as the central automation engine, orchestrating all modules.
- Created a **Tally form interface** where users input a company URL and their "role of interest."
- Integrated **ScrapingBee API** to extract full-page website content.
- Passed the raw content to **Google Gemini Pro** using carefully engineered prompts to return JSON-structured company information (e.g., name, summary, industry, location).
- Developed a **Regex-based Text Parser** to sanitize the AI output and isolate valid JSON strings.
- Used the **Parse JSON module** to extract structured values.
- Automatically inserted extracted data into **Google Sheets**, timestamped and organized.

Key Responsibilities & Skills Applied

- **End-to-End System Design**: Built the entire pipeline using modular, interoperable no-code tools.
- **Tool Integration**: Connected five platforms—Tally, Make.com, ScrapingBee, Google Gemini, and Google Sheets—into a seamless workflow.
- **Prompt Engineering**: Iteratively refined AI prompts to return reliable and valid JSON output under varied inputs.

- **Data Validation & Sanitization**: Developed robust logic using Make.com's Regex-based Text Parser to clean Markdown-wrapped AI responses.
- **Testing & Debugging**: Identified and solved issues with invalid JSON, API credit exhaustion, and non-robust data mappings.

Soft Skills Demonstrated

- **Systems Thinking**: Architected a resilient and modular solution using loosely coupled services.
- **Problem Solving**: Adapted architecture mid-project by replacing OpenAI with Gemini due to free-tier constraints.
- **Iterative Engineering**: Shifted from unreliable text outputs to JSON structuring, then sanitized malformed responses.
- **Creativity in Constraints**: Built an impactful business tool using only free-tier services and without traditional coding.

Hard Skills & Tools Applied

- Platforms: Make.com, Tally Forms, Google Sheets, ScrapingBee, Google Gemini
- Techniques: AI Prompt Engineering, Regex Parsing, Workflow Automation, JSON Structuring, Data Pipeline Validation
- Output Format: Fully structured, timestamped company insights saved row-wise in Sheets

---

Hard Skills Summary

Make.com Automation, Tally Forms, ScrapingBee API, Google Gemini Pro, Prompt Engineering, JSON Parsing, Regex (Text Parser), Google Sheets Integration, No-Code Workflow Design, Data Structuring & Logging, AI-Enhanced Data Extraction

---

Outcome & Future Potential

- **Result**: A robust, one-click research automation tool that transforms an error-prone manual task into a seamless workflow.
- **Impact**: Saves time for job seekers, analysts, and business developers by automating low-value tasks.
- **Next Steps**: Extend functionality with Slack integration, email alerts, multi-page crawling, Google Docs output, and additional AI enrichment.

CV Content Writeup – Self Project

Project Title: Financial Feasibility Dashboard for EV Charging ("PV-Invest")

Duration: July 2025

Role: Full-Stack Developer | Financial Data Analyst | UX Designer

Domain: Renewable Energy Finance | Project Feasibility Analysis | EV Infrastructure | Data Visualization

---

Project Overview

PV-Invest is a fully functional web dashboard designed to empower project developers, financial analysts, and clean energy consultants to rapidly assess the financial viability of hybrid solar + grid electric vehicle (EV) charging stations. Built with **Python** and **Dash**, the application replaces complex spreadsheets with a streamlined, interactive tool. Users configure a wide range of technical and financial parameters to generate a complete year-by-year cash flow analysis, key performance indicators (NPV, IRR), and an environmental impact summary. The project's core output is a professional, multi-page PDF report, designed to support investment decisions and stakeholder presentations.

Key Features & Contributions

- **Financial Modeling Engine:**
    - Engineered the core calculation logic in **Python** to model the entire project lifecycle, including CAPEX (component & infrastructure costs), OPEX (O&M, inflation-adjusted grid costs), and revenue streams (feed-in tariffs, CO_2 price benefits).
    - Implemented a comprehensive, year-wise cash flow statement incorporating depreciation, loan interest, and corporate taxes.
    - Utilized the **NumPy-Financial** library to accurately calculate key investment metrics, including Net Present Value (NPV), Internal Rate of Return (IRR), and the simple payback period.
- **Energy & Emissions Modeling:**
    - Developed a simplified energy balance module using **Pandas** to simulate annual PV generation (with degradation), self-consumption, grid imports, and grid exports based on user-defined system sizes and EV demand.

    - Designed and integrated a CO_2 impact calculator that quantifies the annual tons of carbon saved by comparing the project's emissions against a fully grid-dependent baseline.

- **Interactive Web Dashboard & UI/UX:**
    - Built a clean, responsive, and intuitive single-page application using **Dash Bootstrap Components**, featuring a two-column layout for a clear separation between inputs and outputs.
    - Designed a user-centric control panel with inputs neatly organized into a collapsible **Accordion**, improving clarity and reducing clutter.
    - Implemented a **manual "Run Simulation" button** as a key UX enhancement, giving the user explicit control over when calculations are performed and preventing automatic updates on every input change.
- **Dynamic & Professional PDF Reporting:**
    - Engineered a robust reporting module using **WeasyPrint** to dynamically generate professional, multi-page PDF documents from a styled HTML and CSS template.
    - Solved a critical issue with chart rendering by implementing a pre-processing step to adjust Plotly figure margins, ensuring **un-cropped, complete visualizations** in the final report.
    - Included a summary of all user inputs, high-impact KPI cards, embedded charts, and the **full, formatted financial data table** to create a comprehensive, presentation-ready output.

System Architecture & Technical Stack

- **Language & Framework:** Python, Dash
- **Libraries:** Pandas, NumPy, NumPy-Financial, Plotly, Dash Bootstrap Components
- **Data Visualization:** Interactive charts with Plotly (go.Bar, go.Scatter) rendered in the dashboard and as static images in reports.
- **PDF Generation:** Dynamic HTML template creation with server-side logic, rendered into a polished PDF using WeasyPrint.
- **State Management:** Client-side state handled via a single dcc.Store component, which is updated only when the user manually runs the simulation.

- **Deployment:** Configured for cloud deployment on **Render**, utilizing **Gunicorn** as a production web server and a build.sh script to manage system-level dependencies for WeasyPrint.

Soft Skills Demonstrated

- **Systems Thinking:** Designed the end-to-end data flow, from user input configuration through the backend calculation engine to the dual outputs on the interactive dashboard and the final PDF report.
- **Problem Solving & Debugging:** Iteratively diagnosed and fixed complex deployment issues on a live cloud platform (Render), including dependency conflicts (gunicorn) and library-specific environment setup (WeasyPrint).
- **User-Centric Design:** Significantly refactored the UI based on user feedback, improving the input layout for better readability and implementing the "Run Simulation" button to create a more intentional and responsive user experience.
- **Attention to Detail:** Meticulously styled the final PDF report, addressing issues like chart cropping, font selection, and data formatting to deliver a professional-grade final product.

Hard Skills & Tools Applied

- **Python Full-Stack:** Dash layout design, multi-output callback management, and dcc.Store state handling.
- **Financial Modeling:** Implementation of project finance concepts (CAPEX, OPEX, NPV, IRR, Cash Flow Statements) in a Python environment.
- **Data Visualization:** Time-series plotting with Plotly, creating interactive bar and line charts that clearly communicate financial and environmental results.
- **Dynamic PDF Generation:** Using WeasyPrint, HTML, and CSS to convert dynamic data and Plotly figures into polished, multi-page reports.
- **Cloud Deployment & DevOps:** Successfully deploying a web application with system-level dependencies on a cloud platform (Render), configuring a production server (Gunicorn), and writing shell scripts for the build process.

Hard Skills Summary

Python, Dash, Plotly, Pandas, NumPy, Financial Modeling, NPV, IRR, Data Visualization, UI/UX Design, PDF Generation, WeasyPrint, Dash Bootstrap Components, Cloud Deployment, Render, Gunicorn, HTML, CSS.

Career & Educational Impact

- Demonstrates the ability to build end-to-end, data-driven **decision-support tools** for the renewable energy and project finance sectors.
- Bridges the gap between backend financial logic and an interactive, user-friendly frontend presentation.
- Highlights comfort with the full project lifecycle: from requirement analysis (PRD) and development to debugging, UX refinement, and final cloud deployment.

1. Project Title, Duration, Role, and Domain

Title: PyPSA Code Generator Dashboard
Duration: April 2024 – July 2024
Role: Project Lead & Full Stack Developer
Domain: Energy Systems Modeling, Python Software Development, UI/UX for Digital Energy

---

2. Project Overview

The PyPSA Code Generator Dashboard is a modular, open-source web application designed to democratize power system modeling by providing a no-code interface for configuring and generating PyPSA (Python for Power System Analysis) component scripts. The tool streamlines the creation of energy network models, enabling engineers, consultants, and students to rapidly prototype, validate, and export technically correct PyPSA code blocks without requiring Python coding experience. Developed in response to the steep learning curve and manual errors common in energy system scripting, the dashboard accelerates project setup, reduces onboarding time, and supports transparent, reproducible energy studies for research and industry.

---

3. Key Responsibilities

- **Designed and architected** a tabbed, component-based Streamlit UI supporting all major PyPSA elements (Bus, Load, Generator, StorageUnit, Line, Link, Transformer, Carrier).
- **Developed dynamic forms** with real-time validation, instant code preview, and mode toggling (Basic/Advanced), enhancing both usability and technical accuracy.
- **Implemented modular Jinja2 templates** to map user inputs to valid, production-ready PyPSA Python code for each network component.
- **Integrated documentation and tooltips** to guide users, reduce errors, and lower the barrier for new adopters.
- **Automated session management** to preserve user data across component tabs, ensuring a seamless, multi-step modeling workflow.
- **Led iterative testing and UI refinement** based on feedback from energy professionals and academic peers.

---

4. Results & Impact

- Enabled **100% code generation accuracy** for key PyPSA components, eliminating manual scripting errors and reducing model setup time by over 60%.
- **Enhanced accessibility** for non-programmers, accelerating onboarding for students and consultants in power system studies.

- Supported **rapid scenario prototyping and validation**, empowering users to explore multiple system designs with minimal rework.
- Facilitated **transparent, reproducible modeling workflows** consistent with open-science and industry standards.

---

5. Soft Skills Demonstrated

- Project management and cross-functional collaboration
- User-centered design thinking
- Technical communication and documentation
- Problem-solving and rapid prototyping
- Responsiveness to feedback and iterative improvement

---

6. Hard Skills & Tools Applied

- Streamlit (Python web apps)
- PyPSA (energy modeling framework)
- Jinja2 (template-based code generation)
- Python (core development)
- UI/UX design principles
- Real-time input validation
- Git/GitHub (version control)
- Session and state management
- Automated testing & peer review
- Technical documentation integration

---

7. Hard Skills Summary (Tag Format)

Python, Streamlit, PyPSA, Jinja2, UI/UX Design, Energy Systems Modeling, Real-time Validation, Modular Coding, Code Generation, Session Management, Git, Automated Testing, Technical Documentation, Power Systems, Software Prototyping

🌾 Agri PV Modular Learning Course

Platform: Self-Designed Curriculum | Role: Course Architect & Educator

Completion: July 2025 (Ongoing Updates) | Duration: ~60–80 hours

Mode: Project-based, interdisciplinary, multi-phase, AI-augmented learning framework

📘 Overview

This self-designed, master-level course guides learners from foundational principles to advanced research and innovation in agrivoltaic systems (Agri PV), focusing on the integration of agriculture and photovoltaics. Developed to bridge engineering, agronomy, and policy, it blends theory with real-world application through problem-based modules, simulation labs, and interdisciplinary analysis. The course is structured into four phases: Foundations, Core Techniques, Advanced Methods, and Mastery—emphasizing system-level thinking, hands-on modeling, and AI-enhanced insight.

📚 Core Learning Areas & Topics Covered

- Dual-use land theory: crop physiology vs solar output trade-offs
- Solar PV basics, irradiance modeling (PVWatts, PVGIS)
- Crop simulation using DSSAT under variable shading
- Financial modeling: LCOE, CAPEX/OPEX projections, ROI analysis
- Multi-objective optimization using Pyomo & JuMP (MILP)
- Machine learning for crop yield forecasting and predictive maintenance
- Real-time sensor integration and dashboarding (Grafana, SCADA)
- Climate-policy intersections and sustainability metrics in land use

🛠️ Skills & Application

Applied technical concepts through simulated case studies (e.g., barley yield vs solar gain), optimization labs, and prototyping predictive maintenance systems for Agri PV farms. Integrated PV system design with crop modeling, economic evaluation, and AI workflows—highlighting the energy–agriculture–climate nexus. The course emphasizes problem framing, data preprocessing, and interdisciplinary system design, making it directly applicable to renewable energy innovation, policy analysis, and sustainable infrastructure planning.

🧩 Hard Skills Summary

PVWatts, PVGIS, Python, Pyomo, JuMP, DSSAT, Grafana, Machine Learning, LCOE Analysis, SCADA, System Optimization, TensorFlow

⚡ E-Mobility Modular Learning Course

Platform: Self-Designed Curriculum | Role: Learner & Educator

Completion: July 2025 (Ongoing) | Duration: ~60 hours

Mode: Modular, project-based, theory-to-practice integration

📘 Overview

This comprehensive course provides a structured progression from foundational electrical concepts to advanced innovation in electric mobility. It covers the full E-Mobility ecosystem, including EV components, battery systems, charging infrastructure, and policy frameworks. Through hands-on labs, case studies, and modeling exercises, the course builds analytical and problem-solving skills relevant to research, engineering, and consulting roles in sustainable transportation.

📚 Core Learning Areas & Topics Covered

- Fundamentals of E-Mobility: voltage, power, electric motors, batteries
- Battery chemistry trade-offs, degradation, lifecycle analysis
- Power electronics and drivetrain design
- Charging infrastructure, smart grids, and vehicle-to-grid (V2G) systems
- Mathematical modeling and optimization with Pyomo and JuMP
- Integration of EVs into renewable energy systems
- Policy and market frameworks supporting E-Mobility
- Innovation projects on predictive maintenance and urban charging solutions

🛠️ Skills & Application

Applied theoretical knowledge to practical labs and scenario simulations involving battery modeling, grid interaction, and smart charging network design. Developed expertise in system optimization and data analysis using professional software tools. Gained a comprehensive understanding of socio-technical challenges, preparing for roles in EV system design, consultancy, and policy analysis.

🧩 Hard Skills Summary

Python, Pyomo, JuMP, Battery Modeling, Power Electronics, V2G Systems, Grid Integration, Data Analysis, System Optimization, Policy Frameworks

⚡ Energy Consulting in Germany

Platform: Self-Designed Modular Course | Role: Learner & Curriculum Architect

Completion: July 2025 (Ongoing) | Duration: ~70 hours

Mode: Modular, project-based, industry-aligned with a German market focus

📘 Overview

This comprehensive course equips learners with the technical, regulatory, and consulting skills essential for energy consulting in the German market. Designed from beginner to mastery levels, it combines theoretical knowledge with practical tools, real-world datasets, and policy frameworks to prepare learners for diverse client challenges. Emphasizing problem-solving and communication, it integrates energy system modeling, audit standards, and economic evaluation within Germany's Energiewende context.

📚 Core Learning Areas & Topics Covered

- German energy transition (Energiewende) and institutional landscape (BAFA, BMWK, DENA)
- Energy audits (DIN EN 16247-1, ISO 50001), building physics, load profiling
- Renewable integration, system modeling with OEMOF and Pyomo
- Financial analysis: LCOE, NPV, subsidy frameworks (BEG, EEG, GEG)
- Policy navigation and regulatory impact assessment
- Data analysis and visualization with Python (pandas, matplotlib) and Excel
- Consulting soft skills: client communication, report writing, stakeholder engagement
- Advanced applications: AI in load forecasting, multi-domain energy assessment

🛠️ Skills & Application

Applied frameworks and software to simulate audits, optimize energy systems, and assess regulatory scenarios. Developed client-ready reports and policy briefs integrating technical rigor and economic context. The capstone project reflects real consulting challenges, synthesizing multi-disciplinary knowledge into actionable energy solutions. This course fosters a consultancy mindset focused on adaptability, communication, and technical expertise tailored to Germany's energy market.

🧩 Hard Skills Summary

OEMOF, Pyomo, PVGIS, Python (pandas, matplotlib), Excel, LCOE Analysis, DIN EN 16247-1, ISO 50001, German Energy Policy, Client Communication, AI Load Forecasting

⚙️ Energy System Optimization Course (German Energy Market Focus)

Platform: Self-Designed Curriculum | Role: Learner & Analyst

Completion: July 2025 (Ongoing) | Duration: ~80 hours

Mode: Modular, theory-to-practice, project-driven learning

📘 Overview

This in-depth course offers a structured progression from energy system fundamentals to expert optimization techniques tailored to Germany's liberalized electricity market. Emphasizing system components, market mechanisms, and advanced mathematical programming, the curriculum trains learners to model complex energy systems with uncertainty, integrate emerging technologies, and address regulatory frameworks. Hands-on labs and capstone projects simulate real-world challenges like unit commitment, grid congestion, and sector coupling, preparing learners for roles in energy system design and market strategy.

📚 Core Learning Areas & Topics Covered

- Energy system fundamentals: generation, transmission, load profiles
- German regulatory landscape: Bundesnetzagentur, TSOs, market segments
- Mathematical optimization: linear programming, mixed-integer programming
- Uncertainty handling: stochastic and robust optimization methods
- Sector coupling: power-to-gas, hydrogen, transport integration
- Rolling horizon control and decomposition algorithms
- Market bidding strategies and ancillary service optimization
- Reinforcement learning applications for storage scheduling

🛠️ Skills & Application

Applied advanced optimization methods using Python, Julia, PyPSA, and Calliope for load forecasting, dispatch, and transmission planning. Developed robust, multi-objective models addressing variability in renewables and grid constraints under policy scenarios. Capstone projects synthesized theory into actionable market participation strategies, demonstrating readiness for consulting, grid operations, and energy market analytics in Germany.

🧩 Hard Skills Summary

Python, Julia, PyPSA, Calliope, Pyomo, Linear & Mixed-Integer Programming, Stochastic Optimization, Rolling Horizon, Reinforcement Learning, Energy Market Regulation, Sector Coupling.

DE Mastering German Energy Markets

Platform: Self-Designed Learning Module | Role: Learner & Analyst

Completion: July 2025 | Duration: ~40 hours

Mode: Modular, policy-focused, market analysis

📘 Overview

This course provides a comprehensive exploration of Germany's evolving energy market within the framework of the Energiewende—its ambitious transition to renewable energy dominance. It covers the intricate interplay between policy, regulation, market actors, and renewable integration strategies. Designed to build a strong foundation in the German energy system, it equips learners with critical insights into market liberalization, grid modernization, and the challenges and innovations shaping the future energy landscape.

📚 Core Learning Areas & Topics Covered

- Energiewende policy framework: EEG, EnWG, regulatory roles (BMWK, BNetzA, Bundeskartellamt)
- Market structure: TSOs, DSOs, market operators, and cross-border market coupling
- Renewable energy integration: wind, solar, biomass, and hydropower contributions
- Grid modernization and energy storage solutions for variability management
- Demand response programs and flexible consumption models
- Sector coupling: linking electricity with heating, transport, and industry
- Challenges: grid stability, storage costs, policy coherence
- Future outlook: climate neutrality goals, hydrogen economy, digital grid technologies

🛠️ Skills & Application

Gained a nuanced understanding of Germany's energy market architecture, regulatory environment, and integration of renewables. Developed analytical skills in policy impact assessment and market operation dynamics. This knowledge supports roles in energy consulting, policy analysis, and system planning, particularly within the German or EU energy sectors focused on renewable transitions and market design.

🧩 Hard Skills Summary

German Energy Policy, Market Regulation, Renewable Integration, Grid Modernization, Energy Storage, Sector Coupling, Demand Response, Cross-Border Trading, Policy Analysis

🤖 Machine Learning in Renewable Energy

Platform: Self-Designed Modular Course | Role: Learner & Practitioner

Completion: July 2025 (Ongoing) | Duration: ~50 hours

Mode: Modular, application-focused, interdisciplinary

📘 Overview

This course explores the intersection of machine learning (ML) and renewable energy, equipping learners with techniques to enhance forecasting, optimization, and system integration. Structured in progressive phases, it balances theoretical foundations with practical applications, addressing key challenges such as data quality, model interpretability, and scalability. Learners develop skills to deploy ML solutions for solar and wind forecasting, smart grid management, and predictive maintenance.

📚 Core Learning Areas & Topics Covered

- Supervised, unsupervised, reinforcement, and deep learning methods
- Optimization algorithms: genetic algorithms, simulated annealing
- Solar power and wind energy forecasting with ML models
- Energy storage management and smart grid predictive analytics
- Fault detection and predictive maintenance in renewable assets
- Integration challenges including data quality, scalability, and ethics
- Real-world case studies on AI-driven energy optimization

🛠️ Skills & Application

Applied ML algorithms to energy system problems including generation forecasting, asset management, and grid stability optimization. Developed data-driven models using Python and related ML libraries. Enhanced ability to translate complex data into actionable energy solutions, preparing for roles in data science, energy analytics, and smart grid technology development.

🧩 Hard Skills Summary

Python, Machine Learning, Deep Learning, Reinforcement Learning, Optimization Algorithms, Energy Forecasting, Predictive Maintenance, Smart Grid Analytics, Data Quality Management

🌞 Photovoltaics & Solar Energy

Platform: Self-Designed Modular Course | Role: Learner & Analyst

Completion: July 2025 (Ongoing) | Duration: ~60 hours

Mode: Structured, progressive, hands-on learning

📘 Overview

This course delivers a comprehensive understanding of photovoltaic (PV) technologies, spanning fundamental solar principles to advanced system design and integration. Structured in four phases, it equips learners with theoretical knowledge, practical skills in PV system sizing, performance optimization, economic evaluation, and regulatory navigation. Emphasis on hybrid systems, grid integration, and emerging PV materials prepares learners for innovation and leadership in solar energy projects.

📚 Core Learning Areas & Topics Covered

- Solar radiation and photovoltaic effect fundamentals
- PV system components: modules, inverters, balance-of-system
- System sizing and design for residential and commercial applications
- Performance analysis: shading, orientation, temperature effects
- Economic feasibility: ROI, cost analysis, financial incentives
- Grid integration challenges and smart grid technologies
- Emerging technologies: perovskite cells, hybrid renewable systems
- Research methods and simulation tools (PVsyst, HOMER)

🛠️ Skills & Application

Applied concepts through designing PV systems for real-world scenarios, optimizing system parameters for maximum output, and evaluating economic viability within policy frameworks. Developed skills in simulation and hybrid system integration, fostering capabilities essential for roles in renewable energy engineering, project design, and solar consultancy.

🧩 Hard Skills Summary

Photovoltaic Design, PVsyst, HOMER, System Sizing, Performance Analysis, Economic Evaluation, Grid Integration, Renewable Hybrid Systems, Solar Energy Policy

⏳ Time Series Analysis and Forecasting Models with Python

Platform: Self-Directed Modular Course | Role: Learner & Practitioner

Completion: July 2025 | Duration: ~40 hours

Mode: Theory-practice blend with Python implementations

📘 Overview

This course offers an in-depth journey into time series data analysis and forecasting, emphasizing both theoretical foundations and practical Python applications. Learners develop skills to model temporal data incorporating trend, seasonality, and noise, crucial for sectors such as energy, finance, and operations. The curriculum balances classical statistical models with advanced machine learning techniques, ensuring robust, interpretable, and actionable forecasts aligned with real-world business needs.

📚 Core Learning Areas & Topics Covered

- Time series components: trend, seasonality, residuals
- Data preprocessing: cleaning, imputation, normalization
- Statistical models: ARIMA, Holt-Winters exponential smoothing, seasonal decomposition
- Advanced ML models: XGBoost, LSTM recurrent neural networks
- Feature engineering: lag variables, rolling statistics, calendar effects
- Model evaluation: MAE, RMSE, MAPE, residual diagnostics
- Forecast uncertainty: prediction intervals, bootstrapping, quantile regression
- Time series-specific validation: TimeSeriesSplit and cross-validation methods

🛠️ Skills & Application

Applied statistical and machine learning models using Python libraries like statsmodels, scikit-learn, and tensorflow for energy load forecasting and demand planning. Developed expertise in feature engineering, model tuning, and uncertainty quantification. Cultivated a problem-solving approach that aligns technical rigor with business goals, enabling deployment-ready forecasting solutions.

🧩 Hard Skills Summary

Python, Time Series Analysis, ARIMA, Holt-Winters, LSTM, XGBoost, Feature Engineering, Forecast Evaluation, Prediction Intervals, Data Preprocessing, Model Diagnostics, TimeSeriesSplit

🎨 Comprehensive Modular UI/UX Design Course

Platform: Self-Designed Learning Pathway | Role: Learner & Designer

Completion: July 2025 (Ongoing) | Duration: ~45 hours

Mode: Modular, iterative, hands-on design thinking

📘 Overview

This course guides learners from foundational UI/UX principles to advanced, AI-enhanced design innovations, emphasizing user-centered, ethical, and data-driven approaches. Structured in four progressive phases, it builds expertise in empathic research, prototyping, usability testing, scalable design systems, and emerging interfaces like AR/VR. The curriculum balances conceptual frameworks with practical application using industry-standard tools and real-world case studies, preparing learners to craft inclusive, intuitive digital experiences.

📚 Core Learning Areas & Topics Covered

- User-centered design: personas, user journeys, empathy mapping
- Information architecture and visual design fundamentals
- Wireframing, prototyping, interaction design, usability metrics
- Responsive, accessible design aligned with WCAG standards
- Advanced topics: design systems, UX analytics, voice/AR/VR interfaces
- AI-powered personalization and ethical UX design practices
- Tools & frameworks: Figma, Adobe XD, Maze, TensorFlow.js, Design Thinking

🛠️ Skills & Application

Applied iterative design thinking in labs simulating real product challenges, including interactive prototypes and usability audits. Developed competence in balancing aesthetics with functionality, accessibility compliance, and ethical considerations. Equipped to deliver polished, user-centric solutions and innovate responsibly within multidisciplinary teams.

🧩 Hard Skills Summary

UI/UX Design, Figma, Adobe XD, Wireframing, Prototyping, Usability Testing, Accessibility (WCAG), Design Systems, AI-Powered Personalization, Ethical UX, Interaction Design, Responsive Design

Time Series Analysis: Forecasting Models with Python,	Udemy,	Jul 2025
Learning Data Analytics: 1 Foundations,	LinkedIn Learning,	Mar 2025
Power BI full course,	Learnit Training,	May 2024
Web and Desktop app UI design in Figma,	DesignCode,	Mar 2024
Digital Skills: User Experience,	ACCENTURE,	Feb 2024
UI/UX for Beginner's,	Great Learning Academy,	Feb 2024
AutoCAD 2022 Course - Complete Beginner to Advanced,	Udemy,	Jan 2024
PyQt5 from A-Z,	Udemy,	Aug 2023
Machine Learning with Python,	Cognitive class,	May 2023
Python for Data Science,	Cognitive class,	Apr 2023
PVSyst Training Workshop,	Solar Energy International (SEI),	Oct 2023
"PV*SOL Fundamentals",	Udemy,	
Introduction to Fuel Cells,	Coursera,	Dec 2022
Agile project management,	OPENCLASSROOMS,	Dec 2022
Solar Energy Engineering: Photovoltaic (PV) Technologies,	edX,	Dec 2022

Fuel Cells and Hydrogen Technology,	edX,	Jun 2022
Wind Energy by DTU,	COURSEERA,	Dec 2022
Python for Data Science by IBM,	edX,	Jun 2022
Battery Electric Vehicles,	edX,	May 2022
Fundamentals of Solar Cells,	edX,	Jun 2022
CATIA(V5),	CANTERCADD,	Mar 2020
3D printing,	GRIET,	Aug 2018